GOOGLE | 1POINT3ACRES | TOPIC 10 OF 10 | LESS COMMON TOPIC

Design, Streaming and Scale

1 questions to study here, each on its own page, with the full interview answer and code

18 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

1
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Design, Streaming and Scale | 1 questions | tap a title to jump to it

M 1. Restaurant Waitlist: join / delete / find_first_match **2026-02-01**

SKIP: these fit this topic but have their pages in another topic (18)

M SKIP URL Request Handler Router **topic 5: Trees and Tries**

M SKIP Active Communication Top Users **topic 4: Heaps, Stacks and Queues**

M SKIP Temperature Monitoring Data Structure **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Next-Word Predictor **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Implement Python's itertools.tee **topic 4: Heaps, Stacks and Queues**

M SKIP Union of Sorted Interval Lists **topic 4: Heaps, Stacks and Queues**

M SKIP Streaming Median (LC 295) with Follow-ups **topic 4: Heaps, Stacks and Queues**

M SKIP Batch and Streaming Log Deduplication **topic 4: Heaps, Stacks and Queues**

M SKIP Top-K Frequent in Large Logs **topic 4: Heaps, Stacks and Queues**

M SKIP Website Activity Analytics: Unique Users + Session Time **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Streaming Points: Emit K Within Distance **topic 6: Binary Search and Sorting**

M SKIP First Bad Version with Parallel Search Follow-up **topic 6: Binary Search and Sorting**

M SKIP Employee Hierarchy Add / Delete **topic 5: Trees and Tries**

M SKIP Tic-Tac-Toe System with K Players and 3-in-a-Row Win **topic 8: Grids and Simulation**

M SKIP Streaming Insert + Find K-th Largest **topic 5: Trees and Tries**

M SKIP Run-Length Vector Storage + Dot Product **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Two-Day Log Intersection under Memory Limit **topic 1: Arrays, Hashing and Two Pointers**

H SKIP Babylon Tile-Stack Merge Game **topic 8: Grids and Simulation**

STUDY HERE: topic 10, Design, Streaming and Scale

Site tags: data-structure, oop-design, medium

MEDIUM | 1Point3Acres | onsite-coding | 1 of 1

Restaurant Waitlist: join / delete / find_first_match

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 216/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

QUESTION

Onsite coding round: design and implement a restaurant waitlist data structure supporting join, delete, and find_first_match(table_size). The find call returns the earliest-joined user whose party size fits the given table.

Implement a class with these methods

join(user) — append user (with their party_size) to the waitlist.
delete(user) — remove user from the waitlist (could be anywhere in the queue).
find_first_match(table_size: int) — return the earliest user still on the waitlist whose party_size <= table_size. Do not remove on find.

Implicit constraints

Order of joins matters (FIFO within those that match).
Users may be deleted out of order, so a plain array/linked list with O(1) join cannot give O(1) delete and find together.

EXAMPLES

```
join(Alice, size=2)
join(Bob, size=4)
join(Carol, size=6)
find_first_match(3) -> Alice   // only Alice fits
find_first_match(5) -> Alice   // Alice still earliest
    matching
delete(Alice)
find_first_match(5) -> Bob
find_first_match(2) -> null     // no one fits
```

APPROACH

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

HOW TO THINK ABOUT IT

Join needs the order of arrival, delete can hit anyone, and find must return the earliest person whose party fits the table. Keep one queue per party size and a sequence number per person: find looks at the front of every queue for sizes up to the table size and takes the smallest sequence number. Deletions are marked and skipped lazily when a queue front is read.

Basis: Written for this question (a doubly linked list plus hash map as in LeetCode 146, LRU Cache, with queues bucketed by party size).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
queue = []  # in arrival order
def join(user):
    queue.append(user)
def delete(user):
    queue.remove(user)
def find_first_match(t):
    for user in queue:
        if user.party_size <= t:
            return user
    return None
```

TIME	SPACE
O(n) per find and per delete	O(n)

WHY IT IS SLOW

Delete and find are linear in the list. One queue per party size with lazy deletion makes join and delete O(1) and find touch only the queue fronts.

OPTIMAL SOLUTION · Queue per party size with lazy deletion

TIME	SPACE
O(1) join and delete, O(S) find (S distinct sizes up to the table size)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Restaurant Waitlist: join / delete /
# find_first_match
#
# Onsite coding round: design and implement a restaurant
# waitlist data structure supporting join, delete, and
# find_first_match(table_size). The find call returns the
# earliest-joined user whose party size fits the given
# table. Implement a class with these methods: join(user)
# — append user (with their party_size) to the waitlist.
# delete(user) — remove user from the waitlist (could be
# anywhere in the queue). find_first_match(table_size:
# int) — return the earliest user still on the waitlist
# whose party_size <= table_size. Do not remove on find.
# Implicit constraints Order of joins matters (FIFO within
# those that match). Users may be deleted out of order, so
# a plain array/linked list with O(1) join cannot give
# O(1) delete and find together.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What would you cache to speed up repeated finds?
#
# INTERVIEW NOTES
# Join needs the order of arrival, delete can hit anyone,
# and find must return the earliest person whose party
# fits the table. Keep one queue per party size and a
# sequence number per person: find looks at the front of
# every queue for sizes up to the table size and takes the
# smallest sequence number. Deletions are marked and
# skipped lazily when a queue front is read.
#
# BRUTE FORCE (say it first): Keep one list in join order
# and scan it
# Append on join, remove by value on delete and scan from
# the front on find. Correct and the oracle; delete and
# find are linear. Time O(n) per find and per delete.
# Space O(n).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Keep one list in join order
# and scan it | time O(n) per find and per delete
class WaitlistBrute:
    def __init__(self):
        self.people = []

    def join(self, name, size):
        self.people.append((name, size))

    def delete(self, name):
        self.people = [
            p for p in self.people if p[0] != name
        ]

    def find_first_match(self, table):
        return next(
            (n for n, s in self.people if s <= table),
            None,
        )


# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
        return queue[0] if queue else None

    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
        return best[1] if best else None


if __name__ == "__main__":
    w = Waitlist()
    for name, size in [("Alice", 2), ("Bob", 4)]:
        w.join(name, size)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"
    # find does not remove anyone
    assert w.find_first_match(5) == "Alice"
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    # the same size: arrival order decides
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"
    # deleting someone who is not there is harmless
    w.delete("Nobody")
```

NOTES FROM CANDIDATES

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

PREPARATION

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.